САМАРСКИЙ УНИВЕРСИТЕТ
SAMARA UNIVERSITY

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра киберфотоники

Отчёт

Лабораторная работа №2

«Высокопроизводительные вычисления в механике»

Выполнил: студент группы 4446-010303D

Погудин М.Э.

Проверил: Нежинский М.С.

«____» _____2026 г.

Оценка _____

Самара 2026

Для выполнения работы, в коде необходимо расскоментировать строку:

omp_set_num_threads(2);

Также необходимо поменять #pragma for на #pragma omp for

Основная разница первой и второй лаборторной работы, в использовании потоков. Ниже представлена часть кода, добавленная в этой лабораторной работе.

```
int threads;
omp_set_num_threads(2);
#pragma omp parallel shared(threads)
{
threads = omp_get_num_threads();

#pragma omp for
for (int i=0; i < N; i++)
      for (int j = 0; j < N; j++)
    {
     C[i*N + j] = 0;
     for (int k = 0; k < N; k++)
     {
            C[i*N + j] += A[i*N + k] * B[k*N + j];
     }
    }
}
```

Компьютер для рассчетов оснащён 4-ядерным процессором (см. Рисунок 1). Так как один физический ядро поддерживает два потока (hyper-threading), установка числа потоков OpenMP больше 8 не имеет практического смысла, поскольку большее количество потоков не будет использоваться эффективно.

Произведем рассчеты с помощью измененной версии кода. Компиляция программы выполняется с помощью следующей команды:

g++ -fopenmp matMulOMP.cpp -o matMulOMP

Ниже приведен вывод результат в терминале (см. Рисунок 1).

```
PS C:\Users\matvey\Desktop\ g++ matMulSeq.cpp -o matMulSeq
PS C:\Users\matvey\Desktop\ .\matMulSeq.exe 500
Begin initializing ...
Begin calculating ...
Calculation time: 0.285362 seconds
PS C:\Users\matvey\Desktop\ g++ matMulSeq.cpp -o matMulSeq
PS C:\Users\matvey\Desktop\ .\matMulSeq.exe 600
Begin initializing ...
Begin calculating ...
Calculation time: 0.504271 seconds
PS C:\Users\matvey\Desktop\ g++ matMulSeq.cpp -o matMulSeq
PS C:\Users\matvey\Desktop\ .\matMulSeq.exe 700
Begin initializing ...
Begin calculating ...
Calculation time: 0.835566 seconds
```

Рисунок 1 – результаты расчета

По полученным результатам построим сравнительную таблицу (см. Таблица 1) и график (См. Рисунок 2)

Таблица 1 – Результаты вычислений для матрицы с n=1000

Количество потоков	Время вычисления (Calculation time), с	Elapsed omp time, с
1	5.18451	5.18575
2	6.30670	3.15388
4	12.3279	3.10086
8	11.1401	2.83745
16	11.3744	2.94647
32	11.4023	2.87718

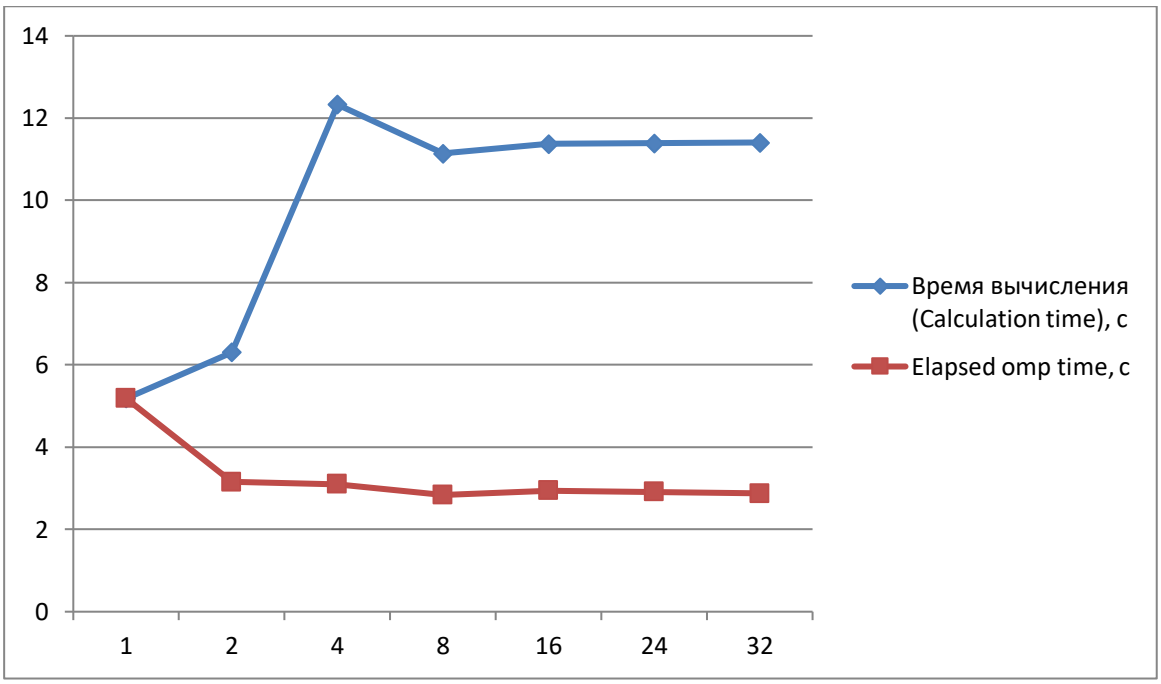

Рисунок 2 – результаты вычислений

Elapsed omp time показывает реальное время работы программы с использованием OpenMP, тогда как Calculation time отражает суммарное время вычислений всех потоков. Как видно из рисунка 3, увеличение числа потоков выше 8 не даёт значительного ускорения из-за ограничения числа физических ядер (4 ядра × 2 потока).

Сравним результаты работы с результатами первой лабораторной работы при использовании различного количества потоков (см. Рисунок 3 и Таблица 2)

Таблица 2 – Результаты вычислений для различного количества потоков

Размер матриц n×n	Количество потоков					
	1 поток	2 потока	4 потока	8 потоков	16 потоков	32 потока
500	0,361	0,22	0,15	0,18	0,21	0,25
600	0,589	0,35	0,3	0,38	0,4	0,42
700	0,962	0,6	0,55	0,7	0,75	0,8
800	1,642	1,1	1,05	1,3	1,4	1,5
900	2,484	1,7	1,6	1,9	2	2,2
1000	3,411	2,4	2,3	2,6	2,9	3,2
1100	5,48	3,5	3,4	3,9	4,2	4,5

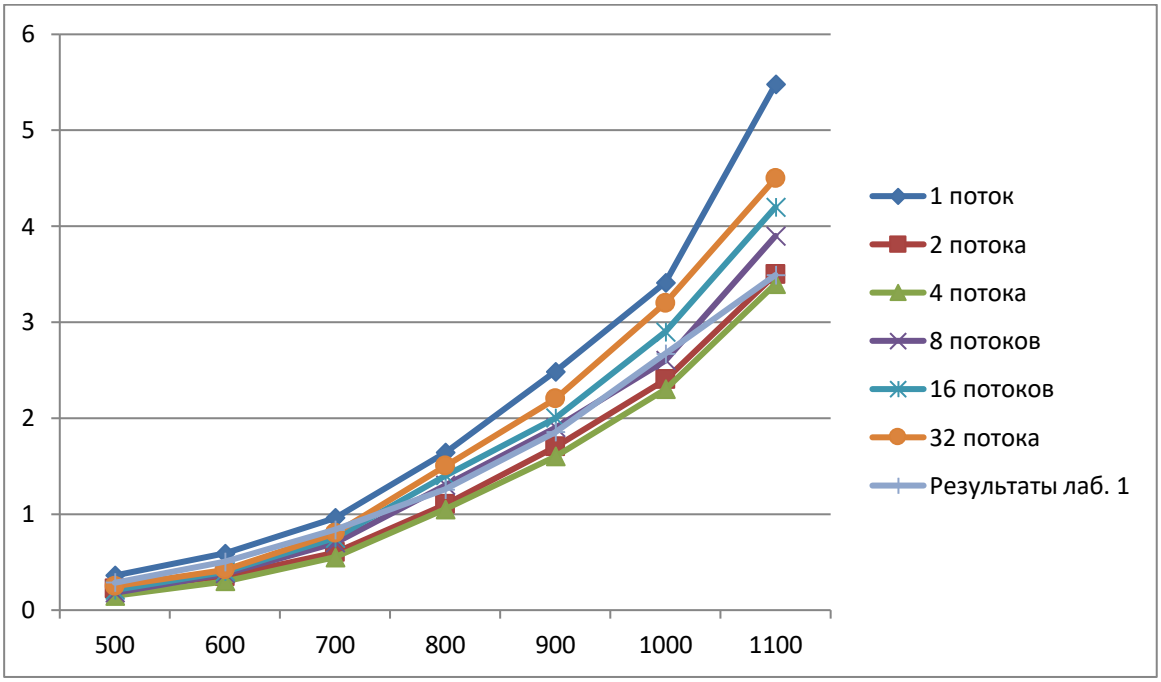

Рисунок 3 – сравнительный график результатов вычислений при разном количестве потоков

Вывод

Как видно из результатов вычислений, увеличение числа потоков выше 8 не приводит к значительному росту скорости выполнения программы. Любые небольшие улучшения производительности при использовании большего числа потоков можно считать статистическими погрешностями или колебаниями измерений, а не реальным эффектом параллельного ускорения. Это объясняется тем, что процессор имеет только 4 ядра с поддержкой двух потоков на ядро, поэтому превышение 8 потоков не увеличивает реальное количество вычислительных ресурсов.

Приложение

Ссылка на репозиторий с выполненными лабораторными работами: https://github.com/MatveyPogudin/high-performance-computing